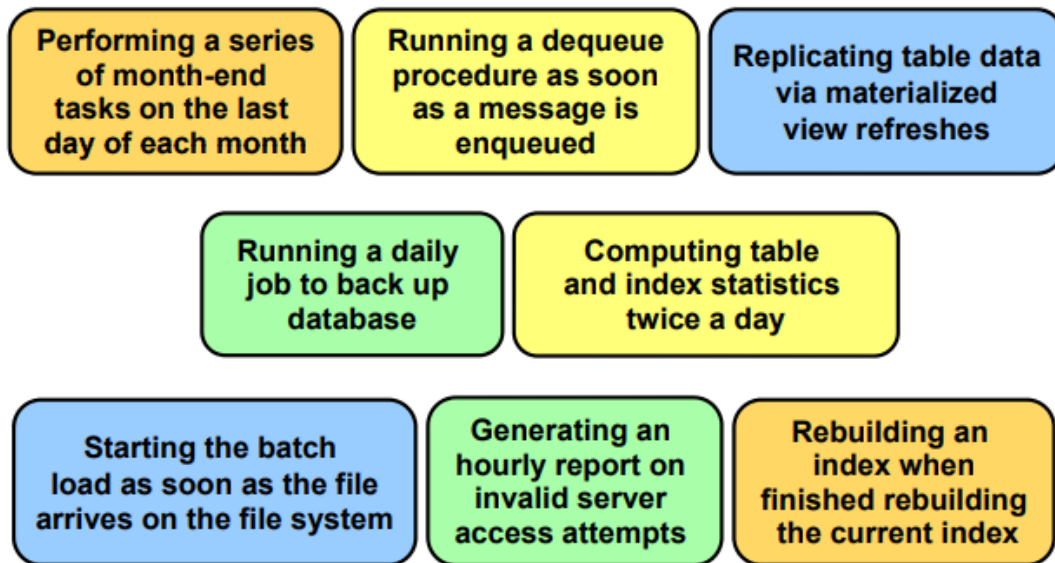


Oracle Database includes Oracle Scheduler, an enterprise job scheduler to help you simplify the scheduling of hundreds or even thousands of tasks. Oracle Scheduler (the Scheduler) is implemented by the procedures and functions in the DBMS_SCHEDULER PL/SQL package.

Simplifying Management Tasks



ORACLE

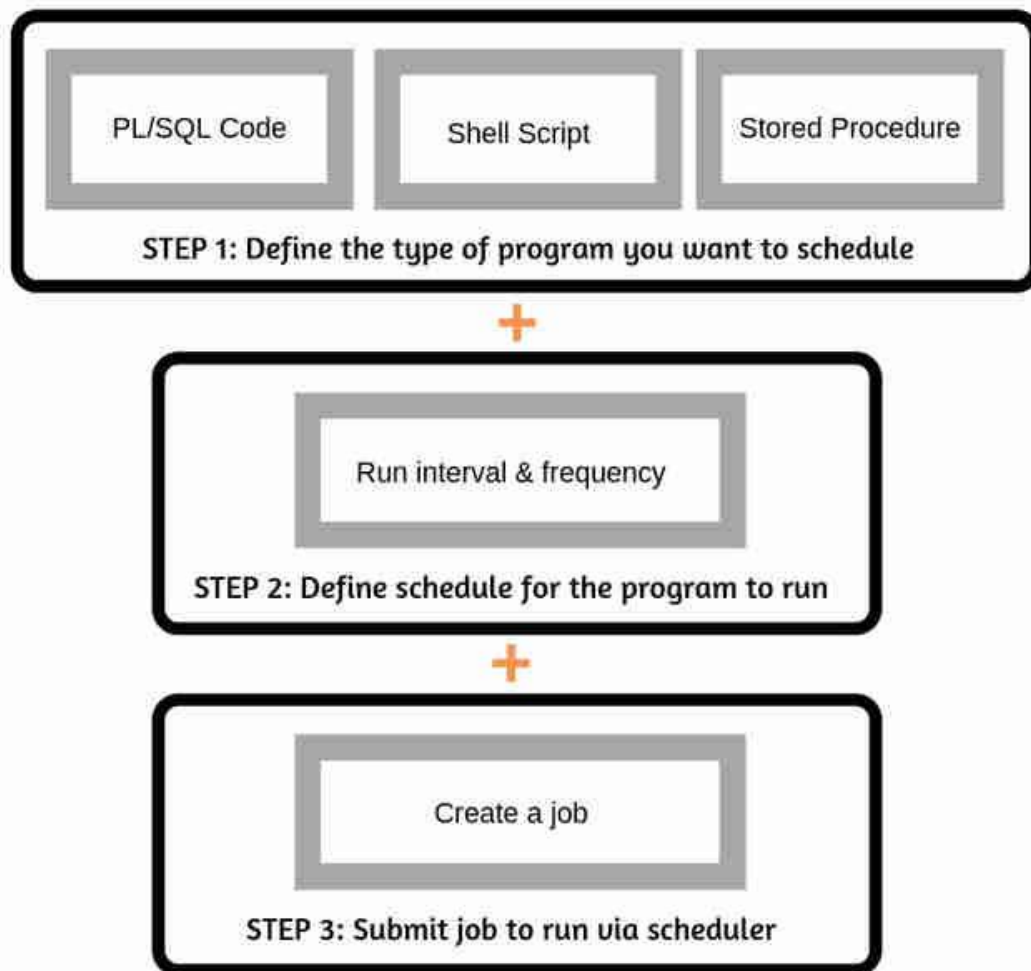
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Before you learn how to schedule jobs via DBMS_SCHEDULER, let us compare it with cron jobs (OS level scripts)

- DBMS_SCHEDULER syntax works same regardless of OS
- Can run jobs based on database events
- If DB is down during a job schedule, it will be executed again once DB starts up
- You can run jobs on remote machines (11gR1 and above)
- Schedule chain of jobs one after another

How DBMS_SCHEDULER works?

Let us first get familiar with different components inside **DBMS_SCHEDULER**. There many components but we will be looking at the most important ones.



As you can see from the above diagram, we need to

- First define a program that is capable of executing PL/SQL script, shell script or stored procedures
- Next, we need to define a schedule for the above program. The schedule contains execution frequency
- Finally, we need to create a job with program name (created in first step) and schedule (created in second step) to DBMS_SCHEDULER

Note: you can directly write a job which includes a program and a schedule. In that case you need not specifically create a program or schedule. But, it is always good to follow standards. Always create a program, followed by schedule and finally job!

Ultimately you will be working with three most important procedures under DBMS_SCHEDULER

- DBMS_SCHEDULER.create_program
- DBMS_SCHEDULER.create_schedule
- DBMS_SCHEDULER.create_job

STEP 1 – Create program

A program tells DBMS_SCHEDULER as to what to execute. It is capable of executing

- PL/SQL Block
- Stored Procedure
- OS level executable file

Sample program to execute a PL/SQL program:

```
BEGIN
  DBMS_SCHEDULER.create_program(
    program_name => 'plsql_program',
    program_type => 'PLSQL_BLOCK',
    program_action => 'BEGIN DBMS_STATS.gather_schema_stats(''HR''); END;',
    enabled => TRUE,
    comments => 'Program to gather HR user statistics');
  DBMS_SCHEDULER.enable (name=>'plsql_program');
END;
```

Sample program to execute a stored procedure

Note: you must define number_of_arguments even before you can enable a program. Notice arguments in the below code must be defined before enabling the program

```
BEGIN
  DBMS_SCHEDULER.create_program(
    program_name => 'stored_procedure_program',
    program_type => 'STORED_PROCEDURE',
    program_action => 'DBMS_STATS.gather_schema_stats',
```

```

number_of_arguments => 1,
enabled => FALSE,
comments => 'Program to gather HR stats using stored procedure');

DBMS_SCHEDULER.define_program_argument(
  program_name => 'stored_procedure_program',
  argument_name => 'ownname',
  argument_position => 1,
  argument_type => 'VARCHAR2',
  default_value => 'SYS');

DBMS_SCHEDULER.enable (name=>'stored_procedure_program');
END;
/
Sample program to execute an OS level executable script file:
BEGIN
  DBMS_SCHEDULER.create_program(
    program_name => 'executable_program',
    program_type => 'EXECUTABLE',
    program_action => '/u02/rman/scripts/db_full_bkp.sh',
    enabled => TRUE,
    comments => 'Program to trigger rman full backup');
  DBMS_SCHEDULER.enable (name=>'executable_program');
END;
/

```

Drop, enable, disable program

To drop a program

```

BEGIN
  DBMS_SCHEDULER.drop_program(program_name=>'plsql_program');
END;
/

```

To enable / disable a program

-- to disable a program

```

BEGIN
  DBMS_SCHEDULER.disable (name=>'plsql_program');
END;
/

```

-- to enable a program

```

BEGIN
  DBMS_SCHEDULER.enable (name=>'plsql_program');
END;
/

```

View program details

You must query [DBA_SCHEDULER_PROGRAMS](#) to view details about scheduled programs

```
set lines 999;
col owner for a20;
col program_name for a30;
SELECT owner, program_name, enabled FROM dba_scheduler_programs where
program_name like '&enter_program_name';
```

STEP 2 – Create schedule

Schedules is what defines DBMS_SCHEDULER when to run a program and at what frequency / interval.

Below is a sample schedule that repeats every hour, exactly at 00 minutes and has no end date

```
BEGIN
DBMS_SCHEDULER.create_schedule (
  schedule_name => 'hourly_sched',
  start_date => SYSTIMESTAMP,
  repeat_interval => 'freq=hourly; byminute=0',
  end_date => NULL,
  comments => 'Run every hour at 00 minutes everyday');
END;
/
```

Drop a schedule

Use below code to drop an existing schedule

```
BEGIN
  DBMS_SCHEDULER.drop_schedule (schedule_name => 'hourly_sched');
END;
/
```

View schedule details

To see schedule details, you must query DBA_SCHEDULER_SCHEDULES view

```
set lines 999;
col schedule_name for a30;
SELECT owner, schedule_name from DBA_SCHEDULER_SCHEDULES;
```

More schedule examples

```
run everyday at midnight
'freq=daily; byhour=0; byminute=0; bysecond=0;'
run everyday at 4 pm
'freq=daily; byhour=16; byminute=0; bysecond=0;'
run every hour at 10 minutes. 1:10, 2:10 .....
'freq=hourly; byminute=10; bysecond=0;'
run every 5 minutes
'freq=minutely; interval=5; bysecond=0;'
run every monday and thursday at 9 pm
'freq=weekly, byday=mon,thu; byhour=21; byminute=0; bysecond=0;'
run friday of each quarter
'freq=monthly; bymonth=1,4,7,10; byday=fri;'
```

STEP 3 – Create job

As mentioned earlier, you need not create program and schedule separately. You can define both program and schedule inside a job. But, it is always good to follow a standard.

DBMS_SCHEDULER is all about jobs. A job consists of a program and a schedule. DBMS_SCHEDULER cannot execute a program or schedule independently. You must create a job and submit to the scheduler.

```
BEGIN
DBMS_SCHEDULER.create_job (
```

```

    job_name => 'test_sched_job',
    program_name => 'plsql_program',
    schedule_name => 'hourly_sched',
    enabled => TRUE,
    comments => 'My test scheduler job');
END;
/

```

Drop, enable, disable job

To drop a job

```

BEGIN
    DBMS_SCHEDULER.drop_job (job_name=>'test_sched_job');
END;
/

```

To enable / disable a job

-- to disable a job

```

BEGIN
    DBMS_SCHEDULER.disable (name=>'test_sched_job');
END;
/

```

-- to enable a job

```

BEGIN
    DBMS_SCHEDULER.enable (name=>'test_sched_job');
END;
/

```

View job details

To see job details, you must query DBA_SCHEDULER_JOBS view

```

set lines 999;
col job_name for a30;
select owner, job_name, enabled from dba_scheduler_jobs;

```

Run jobs manually

Even though you have scheduled jobs inside DBMS_SCHEDULER on a set schedule, you can manually execute jobs too

```
BEGIN
  DBMS_SCHEDULER.run_job (job_name => 'test_sched_job', use_current_session =>
TRUE);
END;
/
```

View job status

To see status of the scheduler job executed previously, you must query

DBA_SCHEDULER_JOB_RUN_DETAILS view

```
select job_name, status, run_duration
from dba_scheduler_job_run_details
where job_name='&enter_job_name';
```

Important Queries

Find the details of the program attached to a job

```
SELECT job_name, enabled, program_name
FROM dba_scheduler_jobs
WHERE job_name LIKE 'TEST%';
```

JOB_NAME	ENABL	PROGRAM_NAME
TEST_SCHED_JOB	TRUE	PLSQL_PROGRAM

Find schedule details attached to a job

```
SELECT job_name, schedule_name, start_date
FROM dba_scheduler_jobs
WHERE job_name like 'TEST%';
```

JOB_NAME	SCHEDULE_NAME	START_DATE
TEST_SCHED_JOB	HOURLY_SCHED	09-APR-19 07.00.33.103403 AM +00:00

Find job current status if it is running or not

```
SELECT JOB_NAME, STATE FROM DBA_SCHEDULER_JOBS
WHERE JOB_NAME LIKE 'TEST%';
```

Below are different job states

- Disabled – Job is disabled
- Scheduled – Job is scheduled to be executed

- | JOB_NAME | STATE |
|----------------|-----------|
| TEST_SCHED_JOB | SCHEDULED |

```
SELECT to_char(log_date, 'DD-MON-YY HH24:MM:SS') TIMESTAMP, job_name, status,
SUBSTR(additional_info, 1, 40) ADDITIONAL_INFO
FROM user_scheduler_job_run_details
WHERE job_name like 'TEST%'
ORDER BY log_date ;
```

[illegible]

- Schedule job execution using the following methods:
 - Time-based scheduling

You can schedule a job to run at a particular date and time, either once or on a repeating basis. You can define complex repeat intervals, such as "every Monday and Thursday at 3:00 a.m. except on public holidays" or "the last Wednesday of each business quarter."

- Event-based scheduling

You can start jobs in response to system or business events. Your applications can detect events and then signal the Scheduler. Depending on the type of signal sent, the Scheduler starts a specific job. Examples of event-based scheduling include starting jobs when a file arrives on a system, when inventory falls below predetermined levels, or when a transaction fails. Beginning with Oracle Database 11g Release 2 (11.2), a Scheduler object called a file watcher simplifies the task of configuring a job to start when a file arrives on a local or remote system.

- Dependency scheduling

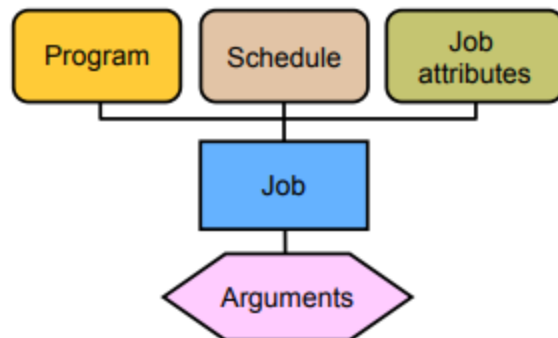
You can set the Scheduler to run tasks based on the outcome of one or more previous tasks. You can define complex dependency chains that include branching and nested chains.

- Prioritize jobs based on business requirements.

The Scheduler provides control over resource allocation among competing jobs, thus aligning job processing with your business needs. This is accomplished in the following ways:

- Controlling Resources by Job Class
 - Controlling Job Prioritization based on Schedules
- Manage and monitor jobs
- Execute and manage jobs in a clustered environment

Oracle Scheduler Core Components



ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Scheduler offers a modularized approach for managing tasks within the Oracle database. By breaking down a task into its components, such as time, location, and database object, the Scheduler offers you an easier way to manage your database environment. The Scheduler uses the following basic components:

- A **job** specifies what needs to be executed. It could be as an example a PL/SQL procedure, a native binary executable, a Java application, or a shell script. You can specify the program (what) and schedule (when) as part of the job definition, or you can use an existing program or schedule instead. You can use arguments for a job to customize its runtime behavior.
- A **schedule** specifies when and how many times a job is executed. A schedule can be based on time or an event. You can define a schedule for a job by using a series of dates, an event, or a combination of the two, along with additional specifications to denote repeating intervals. You can store the schedule for a job separately and then use the same schedule for multiple jobs.
- A **program** is a collection of metadata about a particular executable, script, or procedure. An automated job executes some task. Using a program enables you to modify the job task, or the "what," without modifying the job itself. You can define arguments for a program, enabling users to modify the runtime behavior of the task.

Understanding a Simple Job

```
BEGIN
sys.dbms_scheduler.create_job(
job_name => '"HR"."CREATE LOG TABLE JOB"',
job_type => 'PLSQL_BLOCK', job_action => 'begin
execute immediate (''create table session_history(
snap_time TIMESTAMP WITH LOCAL TIME ZONE,
num_sessions NUMBER)''); end;',
start_date => systimestamp at time zone
'America/New York',
job_class => 'DEFAULT_JOB_CLASS',
comments => 'Create the SESSION_HISTORY table',
auto_drop => FALSE, enabled => TRUE);
END;
```

ORACLE

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

A job has two key components:

- *Action*: "What" needs to be done
- *Schedule*: "When" the action occurs

In the example in the slide, the "what" is enclosed in the top box and the "when" portion is shown in the second box. The "what" is expressed in the `JOB_TYPE` and `JOB_ACTION` parameters. The "when" is expressed in a "schedule," which can be based on time (as shown by the `START_DATE` parameter) or events, or be dependent on the outcome of other jobs.

To use the Scheduler, you create *Scheduler objects*. Schema objects define the what, when, and where for job scheduling. Scheduler objects enable a modular approach to managing tasks.

The principal Scheduler object is the job. A **job** defines the action to perform, the schedule for the action, and the location or locations where the action takes place. Most other scheduler objects are created to support jobs.

Components:

1. Programs

A program object (program) describes what is to be run by the Scheduler.

A program includes:

- An action: For example, the name of a stored procedure, the name of an executable found in the operating system file system (an "external executable"), or the text of a PL/SQL anonymous block.
- Type: `STORED_PROCEDURE`, `PLSQL_BLOCK`, `SQL_SCRIPT`, `EXTERNAL_SCRIPT`, `BACKUP_SCRIPT`, or `EXECUTABLE`, where `EXECUTABLE` indicates an external executable.
- Number of arguments: The number of arguments that the stored procedure or external executable accepts.

A program is a separate entity from a job. A job runs at a certain time or because a certain event occurred, and invokes a certain program. You can create jobs that point to existing program objects, which means that different jobs can use the same program and run the program at different times and with different settings. With the right privileges, different users can use the same program without having to redefine it.

2.Schedules

A schedule object (schedule) specifies when and how many times a job is run.

Schedules can be shared by multiple jobs. For example, the end of a business quarter may be a common time frame for many jobs. Rather than defining an end-of-quarter schedule each time a new job is defined, job creators can point to a named schedule.

There are two types of schedules:

- time schedules

With time schedules, you can schedule jobs to run immediately or at a later time. Time schedules include a start date and time, optional end date and time, and optional repeat interval.

- event schedules

With event schedules, you can specify that a job executes when a certain event occurs, such as inventory falling below a threshold or a file arriving on a system.

3.Jobs

A job describes a user-defined task.

A job object (job) is a collection of metadata that describes a user-defined task. It defines what must be executed (the action), when (the one-time or recurring schedule or a triggering event), where (the destinations), and with what credentials. A job has an owner, which is the schema in which it is created.

A job that runs a database program unit is known as a **database job**. A job that runs an external executable is known as an **external job**.

Specifying a Job Action

You can specify a job action by specifying the database program unit or external executable to be run or the name of an existing program object (program).

Specifying a Job Schedule

You can specify a job schedule by setting attributes of the job object or the name of an existing schedule object (schedule).

You specify the job schedule in one of the following ways:

- By setting attributes of the job object to define **start and end dates and a repeat interval**, or to define an event that starts the job. This is known as specifying the schedule **inline**.
- By specifying as a job attribute the name of an existing schedule, which defines start and end dates and a repeat interval, or defines an event.

After you create a job and enable it, the Scheduler automatically runs the job according to its schedule or when the specified event is detected. You can view the run status of job and its job log by querying data dictionary views. If a job runs on multiple destinations, you can query the status of the job at each destination.

Destinations

You can specify external and database destinations for running a job.

A destination object (destination) defines a location for running a job.

There are two types of destinations:

- External destination: Specifies a remote host name and IP address for running a remote external job.
- Database destination: Specifies a remote database instance for running a remote database job.

Jobs that run external executables (external jobs) must specify external destinations, and jobs that run database program units (database jobs) must specify database destinations.

If you specify a destination when you create a job, the job runs on that destination. If you do not specify a destination, the job runs locally, on the system on which it is created.

You can also create a destination group, which consists of a list of destinations, and reference this destination group when creating a job. In this case, the job runs on all destinations in the group.

Job Classes

Job classes enable you to assign the same attributes to member jobs, set resource allocation for member jobs, and group jobs for prioritization.

You typically create job classes only when you are in the role of Scheduler administrator.

Job classes provide a way to:

- Assign the same set of attribute values to member jobs

Each job class specifies a set of attributes, such as logging level. When you assign a job to a job class, the job inherits those attributes. For example, you can specify the same policy for purging log entries for all payroll jobs.

- Set service affinity for member jobs

You can set the `service` attribute of a job class to a desired database service name. This determines the instances in a Real Application Clusters environment that run the member jobs, and optionally, the system resources that are assigned to member jobs.

- Set resource allocation for member jobs

Job classes provide the link between the Database Resource Manager and the Scheduler, because each job class can specify a resource consumer group as an attribute. Member jobs then belong to the specified consumer group and are assigned resources according to settings in the current resource plan.

Alternatively, you can leave the `resource_consumer_group` attribute NULL and set the `service` attribute of a job class to a desired database service name. That service can in turn be mapped to a resource consumer group. If both the `resource_consumer_group` and `service` attributes are set, and the designated service maps to a resource consumer group, the resource consumer group named in the `resource_consumer_group` attribute takes precedence.

- Group jobs for prioritization

Within the same job class, you can assign priority values of 1-5 to individual jobs so that if two jobs in the class are scheduled to start at the same time, the one with the higher priority takes precedence. This ensures that you do not have a less important job preventing the timely completion of a more important one.

If two jobs have the same assigned priority value, the job with the earlier start date takes precedence. If no priority is assigned to a job, its priority defaults to 3.

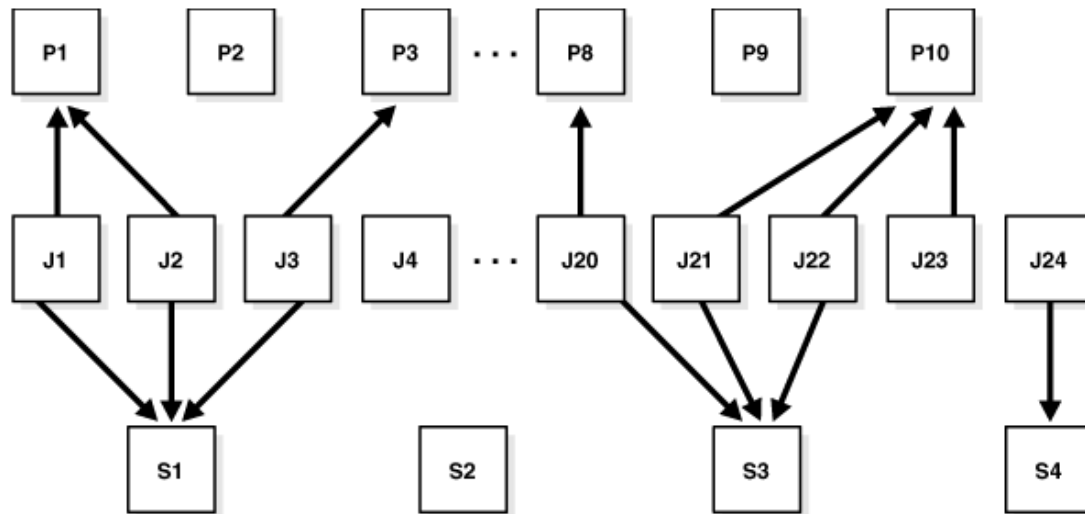
When defining job classes, try to classify jobs by functionality. Consider dividing jobs into groups that access similar data, such as marketing, production, sales, finance, and human resources.

Some of the restrictions to keep in mind are:

- A job must be part of exactly one class. When you create a job, you can specify which class the job is part of. If you do not specify a class, the job automatically becomes part of the class `DEFAULT_JOB_CLASS`.
- Dropping a class while there are still jobs in that class results in an error. You can force a class to be dropped even if there are still jobs that are members of that class, but all jobs referring to

that class are then automatically disabled and assigned to the class `DEFAULT_JOB_CLASS`. Jobs belonging to the dropped class that are already running continue to run under class settings determined at the start of the job.

Figure 28-5 Relationships Among Programs, Jobs, and Schedules



Access Rights

Table 29-1 Job Tasks and Their Procedures

Task	Procedure	Privilege Needed
Create a job	CREATE_JOB or CREATE_JOBS	CREATE JOB or CREATE ANY JOB
Alter a job	SET_ATTRIBUTE or SET_JOB_ATTRIBUTES	ALTER or CREATE ANY JOB or be the owner
Run a job	RUN_JOB	ALTER or CREATE ANY JOB or be the owner
Copy a job	COPY_JOB	ALTER or CREATE ANY JOB or be the owner
Drop a job	DROP_JOB	ALTER or CREATE ANY JOB or be the owner
Stop a job	STOP_JOB	ALTER or CREATE ANY JOB or be the owner
Disable a job	DISABLE	ALTER or CREATE ANY JOB or be the owner
Enable a job	ENABLE	ALTER or CREATE ANY JOB or be the owner

Oracle Scheduling Scheduler Privileges

Oracle Tips by Burleson Consulting

Scheduler Privileges

Table 2.1 shows the system privileges associated with the scheduler.

SYSTEM PRIVILEGE	PURPOSE
CREATE JOB	Enables the user to create jobs, schedules and programs in their own schema. The user can always alter and drop jobs, schedules and programs which they own, even when they do not have the CREATE JOB privilege.
CREATE ANY JOB	Enables the user to create jobs, schedules, and programs in any schema. This effectively gives the grantee the ability to run code as any user so it must be issued with care.
EXECUTE ANY PROGRAM	Enables jobs the ability to use programs from any schema.
EXECUTE ANY CLASS	Enables jobs to run under any job class.
MANAGE SCHEDULER	Enables the user to create, alter and drop job classes, windows and window groups. It also enables the user to purge scheduler logs and modify scheduler attributes.

Table 2.1 - System privileges associated with the scheduler.

Table 2.2 shows the object privileges associated with the scheduler.

OBJECT PRIVILEGE	PURPOSE
EXECUTE	This can only be granted for programs and job classes, enabling jobs to use the granted objects.
ALTER	Enables the user to alter or drop the object it is granted on. The alter operations vary depending on the object.
ALL	The can be granted on jobs, programs, schedules and job classes. It enables the user to perform all possible operations on the object it is granted on.

Table 2.2 - The object privileges associated with the scheduler.

The `SCHEDULER_ADMIN` role has all the above privileges granted to it with the `ADMIN` option, making it extremely powerful. This role is granted in turn to the `DBA` with the `ADMIN` option.

Scheduling Jobs with Oracle Scheduler

You can create, run, and manage jobs with Oracle Scheduler.

A job is the combination of a schedule and a program, along with any additional arguments required by the program.

A job is the combination of a schedule and a program, along with any additional arguments required by the program.

You create one or more jobs using the `DBMS_SCHEDULER.CREATE_JOB` or `DBMS_SCHEDULER.CREATE_JOBS` procedures

You must have the `CREATE JOB` privilege to create a job in your own schema, and the `CREATE ANY JOB` privilege to create a job in any schema except `SYS`.

For each job being created, you specify a job type, an action, and a schedule. You can also optionally specify a credential name, a destination or destination group name, a job class, and other attributes. As soon as you enable a job, it is automatically run by the Scheduler at its next scheduled date and time. By default, jobs are disabled when created and must be enabled with `DBMS_SCHEDULER.ENABLE` to run. You can also set the `enabled` argument of the `CREATE_JOB` procedure to `TRUE`, in which case the job is ready to be automatically run, according to its schedule, as soon as you create it.

You can create a job in another schema by specifying `schema.job_name`. The creator of a job is, therefore, not necessarily the job owner. The job owner is the user in whose schema the job is created.

The following example demonstrates creating a database job called `update_sales`, which calls a package procedure in the `OPS` schema that updates a sales summary table:

```
BEGIN
  DBMS_SCHEDULER.CREATE_JOB (
    job_name          => 'update_sales',
    job_type          => 'STORED_PROCEDURE',
    job_action        => 'OPS.SALES_PKG.UPDATE_SALES_SUMMARY',
```

```
start_date      => '28-APR-08 07.00.00 PM Australia/Sydney',
repeat_interval => 'FREQ=DAILY;INTERVAL=2', /* every other day */
end_date        => '20-NOV-08 07.00.00 PM Australia/Sydney',
auto_drop       => FALSE,
job_class       => 'batch_update_jobs',
comments        => 'My new job');

END;

/
```

Because no `destination_name` attribute is specified, the job runs on the originating (local) database. The job runs as the user who created the job.

The `repeat_interval` argument specifies that this job runs every other day until it reaches the end date and time. Another way to limit the number of times that a repeating job runs is to set its `max_runs` attribute to a positive number.

The job is disabled when it is created, by default. You must enable it with `DBMS_SCHEDULER.ENABLE` before the Scheduler will automatically run it.

Jobs are set to be automatically dropped by default after they complete. Setting the `auto_drop` attribute to `FALSE` causes the job to persist. Note that repeating jobs are not auto-dropped unless the job end date passes, the maximum number of runs (`max_runs`) is reached, or the maximum number of failures is reached (`max_failures`).

After a job is created, it can be queried using the `*_SCHEDULER_JOBS` views.